

Chapter 17. Sequential Recommenders

In our journey so far, you've learned about a variety of features that appear as explicit or as latent components in the recommendation problem. One kind of feature, which has appeared implicitly, is the history of previous recommendations and interactions. You may wish to protest here: "All of the work we've done so far considers the previous recommendations and interactions! We've even learned about prequential training data."

That is true, but it fails to account for more explicit relationships between the *sequence of recommendations leading up to the inference request*. Let's look at an example to distinguish the two. Your video-streaming website knows that you've previously seen all of Darren Aronofsky's films, so when *The Whale* is released, the website is very likely to recommend it. But this type of recommendation is different from one you might receive after finishing episode 10 of *Succession*. You may have been watching Aronofsky films over a long time period—*Pi* many years ago and *Black Swan* earlier this year. But you have been watching an episode of *Succession* each night this week, and your entire recent history is made up of Logan Roy. This latter example is a sequential recommendation problem: using the most recent ordered list of interactions to predict what you'll enjoy next.

In terms of the modeling objective, the recommenders we've seen use pairwise relationships between potential recommendations and historical interactions. Sequential recommendation aims to predict users' next actions based on the sequential interactions in the past that may be of much higher *order*—i.e., combinations of interactions among three or more items. Most sequential recommendation models involve sequential data-mining techniques such as Markov chains, recurrent neural networks (RNNs), and self-attention. These models usually take into consid-

eration short-term user behavior and are less sensitive, even oblivious, to the global user preferences that have stabilized over time.

Initial work in sequential recommendations focused on modeling the transitions between successive items. These used Markov chains and translation-based methods. As deep learning methods showed more and more promise in modeling sequential data—such as their biggest success in NLP—there have been many attempts to use neural network architectures to model sequential dynamics of a user’s interaction history. Early successes in this direction include GRU4Rec using an RNN to model users’ sequential interactions. Recently, transformer architectures have demonstrated superior performance for sequential data modeling. The transformer architecture lends itself to efficient parallelization and is effective at modeling long-range sequences.

Markov Chains

Despite mining for relationships to historical recommendations, the models we’ve been considering often fail to capture sequential patterns in user behavior, thereby disregarding the chronological order of user interactions. To address this shortcoming, sequential recommender systems were developed, incorporating techniques like Markov chains to model the temporal dependencies between items.

A *Markov chain* is a stochastic model that operates on the principle of *memorylessness*. It models the probability of transitioning from one state to another—given the current state—without considering the sequence of preceding events. Markov chains model the sequential behavior of users by considering each state as an item, and the transition probabilities as the likelihood of a user interacting with a certain item after the current one.

The first-order Markov chain, in which the future state depends solely on the current state, was a common strategy in early sequential recommenders. Despite its simplicity, the first-order Markov chain is effective in capturing short-term, item-to-item transition patterns, improving the quality of recommendations over nonsequential methods.

Take, for example, our preceding *Succession* example. If you're using only a first-order Markov chain, a really great heuristic would be "What is the next episode in the series, if it's a series; otherwise, fall back on a collaborative filtering (CF) model." You can see that for a huge percentage of watch hours, this naive first-order chain would simply tell the user to watch the next episode of a show. Not particularly enlightening, but a good sign. When you abstract this out further, you start to get more powerful methods.

The first-order assumption does not always hold in real-world applications, as user behavior is often influenced by a longer history of interactions. To overcome this limitation, higher-order Markov chains look further back: the next state is determined by a set of previous states, providing a richer model of user behavior. Nevertheless, it's crucial to select the appropriate order, as too high an order may lead to overfitting and sparsity of the transition matrix.

Order-Two Markov Chain

Let's consider an example of an *order-two Markov chain* model using the weather. Assume we have three states: sunny (S), cloudy (C), and rainy (R).

In an order-two Markov chain, the weather of today (t) would depend on the weather of yesterday ($t - 1$) and the day before yesterday ($t - 2$). The transition probability can be denoted as $P(S_t|S_{t-1}, S_{t-2})$.

The Markov chain can be defined by a transition matrix that provides the probabilities of transitioning from one state to another. However, because we're dealing with an order-two Markov chain, we would have a transition tensor instead. For simplicity, let's say we have the following transition probabilities:

$$\begin{aligned} P(S|S, S) &= 0.7, P(C|S, S) = 0.2, P(R|S, S) = 0.1, \\ P(S|S, C) &= 0.3, P(C|S, C) = 0.4, P(R|S, C) = 0.3, \\ &\dots \end{aligned}$$

You can visualize these probabilities in a three-dimensional cube. The first two dimensions represent the state of today and yesterday, and the

third dimension represents the possible states of tomorrow.

If the weather was sunny for the last two days and we want to predict the weather for tomorrow, we would look at the transition probabilities starting with (S, S) , which are $P(S|S, S) = 0.7$, $P(C|S, S) = 0.2$, and $P(R|S, S) = 0.1$. Therefore, according to our model, there's a 70% chance that it will be sunny, a 20% chance that it will be cloudy, and a 10% chance that it will be rainy.

The probabilities in the transition matrix (or tensor) are typically estimated from data. If you have a historical record of the weather for several years, you can count the number of times each transition occurs and divide by the total number of transitions to estimate the probability.

This is only a basic demonstration of an order-two Markov chain. In real applications, the states might be much more numerous and the transition matrix much larger, but the principles remain the same.

Other Markov Models

A more advanced Markovian approach is the *Markov decision process* (MDP), which extends the Markov chain by introducing actions and rewards. In the context of recommender systems, each action could represent a recommendation, and the reward could be the user's response to the recommendation. By incorporating user feedback, the MDP can learn more personalized recommendation strategies.

MDPs are defined by a tuple (S, A, P, R) , where S is the set of states, A is the set of actions, P is the state transition probability matrix, and R is the reward function.

Let's use a simplified MDP for a movie recommender system as an example:

States (S)

These could represent the genres of movies a user has watched in the past. For simplicity, let's say we have three states: Comedy (C),

Drama (D), and Action (A).

Actions (A)

These could represent the movies that can be recommended. For this example, let's say we have five actions (movies): Movies 1, 2, 3, 4, and 5.

Transition probabilities (P)

This represents the likelihood of transitioning from one state to another, given a specific action. For instance, if the user just watched a Drama (D) and we recommend Movie 3 (which is an Action movie), the transition probability $P(A|D, \text{Movie3})$ might be 0.6, indicating a 60% chance the user will watch another Action movie.

Rewards (R)

This is the feedback from the user after taking an action (recommendation). Let's assume for simplicity that a user's click on a recommended movie gives a reward of +1 and no click is a reward of 0.

The aim of the recommender system in this context is to learn a policy $\pi : S \rightarrow A$ that maximizes the expected cumulative reward. A policy dictates which action the agent (the recommender system) should take in each state.

This policy can be learned via reinforcement learning algorithms, such as Q-learning or policy iteration, which essentially learn the value of taking an action in a state (i.e., recommending a movie after the user has watched a certain genre), considering the immediate reward and the potential future rewards.

The main challenge in a real-world recommender system scenario is that both the state and action spaces are extremely large, and the transition dynamics and reward function can be complex and difficult to estimate accurately. But, the principles demonstrated in this simple example remain the same.

Despite the promising performance of Markov chain-based recommender systems, several challenges remain. The *memorylessness* assumption of the Markov chain may not hold in certain scenarios where long-term dependencies exist. Furthermore, most Markov chain models treat user-item interactions as binary events (either interaction or no interaction), which oversimplifies the variety of interactions users may have with items, such as browsing, clicking, and purchasing.

Next, we'll cover neural networks. We'll see how some architectures you're likely familiar with can be relevant to learning a sequential recommender task.

RNN and CNN Architectures

Recurrent neural networks (RNNs) are a type of neural network architecture designed to recognize patterns in sequences of data, such as text, speech, or time series data. These networks are *recurrent* in that the outputs from one step in the sequence are fed back into the network as inputs while processing the next step. This gives RNNs a form of memory, which is helpful for tasks like language modeling, where each word depends on the previous words.

At each time step, an RNN takes in an input (like a word in a sentence) and produces an output (like a prediction of the next word). It also updates its internal state, which is a representation of what it has “seen” last in the sequence. This internal state is passed back into the network when processing the next input. As a result, the network can use information from previous steps to influence its predictions for the current step. This is what allows RNNs to effectively process sequential data.

[GRU4Rec](#) used recurrent neural networks to model session-based recommendations in one of the first applications of neural network architectures to the recommendation problem. A *session* refers to a single contiguous period of user interaction, like time spent on a page without the user navigating away or turning off their computer.

Here we will see a dramatic advantage of sequential recommendation systems: most traditional recommendation methods rely on an explicit user ID to build a user-interest profile. However, session-based recommendations operate over anonymous user sessions that are often quite short to allow for a profile modeling. Moreover, a lot of variance can occur in user motivations in different sessions. A solution via user-agnostic recommendation that works for such recommendation situations is an item-based model in which an item-item similarity matrix is calculated based on items co-occurring within a single session. This precomputed similarity matrix is employed at runtime to recommend the most similar item to the one last clicked. This approach has obvious limitations such as relying only on the last clicked item. To this end, GRU4Rec uses all the items in the session and models the session as a sequence of items. The task of recommending items to be added translates to the prediction of the next item in the sequence.

Unlike the small fixed-size vocabulary of languages, recommendation systems are required to reason over a large number of items that grows over time as more items are added. To handle this concern, pairwise ranking losses (e.g., BPR) are considered. GRU4Rec is further extended in [GRU4Rec+](#), which utilizes a new loss function specifically designed for gains in top- k recommendation. These loss functions blend deep learning and LTR to address neural recommendation settings.

A different approach to neural networks for recommendations adopted CNNs for sequential recommendation. We won't cover the basics of CNNs here, but you can consult [“How Do Convolutional Neural Networks Work?”](#) by Brandon Rohrer for the essentials.

Let's discuss one method that has shown a lot of success, [CosRec](#), as visualized in [Figure 17-1](#). This method (and others) starts with a structure similar to that of our MF used throughout most of the book: a user-item matrix. We assume that there are two latent factor matrices, E_U and E_I , but let's first focus on the item matrix.

Each vector in the item matrix is an embedding vector for a single item, but we wish to encode sequences: take sequences of length L and collect

those embedding vectors. We now have an $L \times D$ matrix with a row per each item in the sequence. Take adjacent rows as pairs and concatenate them for each vector in a three-tensor; this effectively captures the sequence as a series of pairwise transitions. This three-tensor can be passed through a vectorized 2D CNN to yield a vector (of length L) that is concatenated with the original user vector and fed through a fully connected layer. Finally, binary cross-entropy is our loss function to attempt to predict the best recommendation.

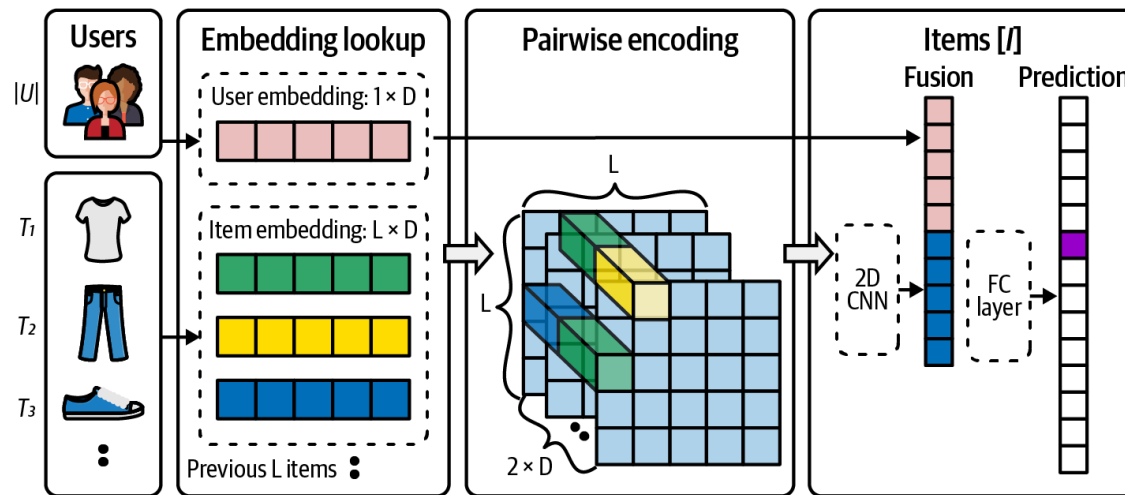


Figure 17-1. CosRec CNN

Attention Architectures

A term that is commonly associated with neural networks and that may ring a bell for you by now is *attention*. This is because transformers, in particular the kind that appear in large language models (LLMs) like the generalized pretrained transformer, have become a central focus among AI users.

We will give an extremely brief, and less technical, introduction to self-attention and the transformer here. For a more complete guide on transformers, consult the excellent overview in [“Transformers from Scratch”](#) by Brandon Rohrer.

First, let’s state the key differentiating assumption about a transformer model: the embeddings are positional. We’re hoping to learn not only one embedding for every item but also an embedding for every item-position

pair. Therefore, when an article is the first in a session and the last in a session, those two instances are treated as *two separate items*.

Another important notion is stacking. When building transformers, we often think of the architecture as a layer cake, with sections stacked on top of one another. The key components are the embeddings, the self-attention layer, the skip-addition, and the feed-forward layer. The most complicated operations happen in self-attention, so let's focus on that first. We just discussed the positional embeddings, which are sent as a sequence of these embedding vectors; recall that a transformer is a sequence-to-sequence model! The skip-addition means that we push the embedding forward *around* the self-attention layer (and the feed-forward layer above) and add it to the positional output of the attention layer. The feed-forward layer is an unexciting multilayer perceptron that stays in the positional columns and uses a ReLU or GeLU activation.

RELU VERSUS GELU

ReLU (Rectified Linear Unit) is an activation function defined as $f(x) = \max(0, x)$. GeLU (Gaussian Error Linear Unit) is another activation function approximated as $f(x) = 0.5x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} x + 0.044715x^3 \right) \right)$, inspired by the Gaussian cumulative distribution function. The intuition behind GeLU is that it tends to allow small values of x to pass through while smoothly saturating extreme values, potentially enabling better gradient flow for deep models. Both functions introduce nonlinearity in neural networks, with GeLU often demonstrating improved learning dynamics over ReLU in certain contexts.

Here are some quick tips on self-attention:

- The idea behind self-attention is that everything in the sequence affects everything else, in some manner.
- The self-attention layer is learning four weight matrices per head.
- The heads are in 1-1 correspondence with the sequence length.
- We often call the weight matrices Q, K, O, V . Both Q and K get crossed with the positional embedding, but O and V are first crossed into an embedding-dimension-sized square matrix before dotting

with the embedding. $Q\dot{E}$ and $K\dot{E}$ multiply to create the eponymous *attention* matrix, over which we take a row-wise softmax to get the attention vector.

- Some normalizations exist, but we'll disregard them as inessential for understanding.

When we want to speak accurately but briefly about attention, we usually say, “It takes a sequence of positional embeddings and mashes them all together to learn how they're related.”

Self-Attentive Sequential Recommendation

[SASRec](#) is the first transformer model we'll consider. This autoregressive sequential model (similar to a causal language model) predicts the next user interaction from past user interactions. Inspired by the success of the transformer models in sequential mining tasks, the self-attention-based architecture is used for sequential recommendation.

When we say that the SASRec model is trained in an autoregressive manner, we mean that the self-attention is allowed to attend to only the earlier positions in the sequence; looking into the future is not permitted. In terms of the mashing we referenced earlier, think of this as only mashing forward the influence. Some people call this “causal” because it respects the causal arrow of time. The model also allows for a learnable positional encoding, which means that the updates carry down to the embedding layer. This model uses two transformer blocks.

BERT4Rec

Inspired by the BERT model in NLP, [BERT4Rec](#) improves upon SASRec by training a bidirectional masked sequential (language) model.

While BERT uses a masked language model for pretraining word embeddings, BERT4Rec uses this architecture to train end-to-end recommendation systems. It tries to predict the masked items in the user-interaction sequence. Similar to the original BERT model, the self-attention is bidirectional: it can look at both past and future interactions in the action se-

quence. To prevent leakage of future information and to emulate the realistic settings, only the last item in the sequence is masked during inference. Using item masking, BERT4Rec outperforms SASRec. However, a drawback of the BERT4Rec model is that it is quite compute intensive and requires much more training time.

Recency Sampling

Sequential recommendation and the adoption of transformer architecture in these tasks has seen a lot of interest recently. These deep neural network models like BERT4Rec and SASRec have shown improved performance over traditional approaches. However, these models suffer from slow training problems. A recently published paper—ha ha, get it—addresses the question of improving training efficiency while achieving state-of-the-art performance. See [“Effective and Efficient Training for Sequential Recommendation Using Recency Sampling”](#) by Aleksandr Petrov and Craig Macdonald for details.

The two training paradigms we’ve just described for sequential models are autoregressive, which tries to predict the next item in the user-interaction sequence, and masked, which tries to predict masked items in the interaction sequence. The autoregressive approach doesn’t use the beginning of the sequence as labels in the training process, and thus valuable information is lost. The masked approach, on the other hand, is only weakly related to the end goal of the sequential recommendation.

The paper by Petrov and Macdonald proposes a recency-based sampling of positive examples from the sequences to build the training data. The sampling is designed to give more recent interactions higher chances of being sampled. However, because of the probabilistic nature of the sampling mechanism, even the oldest of the interactions have nonzero chances of being chosen. An exponential function is employed as a sampling routine that interpolates between the masking-based sampling, where each interaction has equal probability of being sampled, and autoregressive sampling, where items from the end of the sequence are sampled. This showed superior performance in sequential recommendation tasks while requiring much less training time. Compare this approach to

some of the other examples where we saw sampling provide significant improvements in training recommender systems!

Merging Static and Sequential

Pinterest recently released [“Rethinking Personalized Ranking at Pinterest: An End-to-End Approach”](#) by Jiajing Xu et al. describing its personalized recommendation system, which is built to leverage raw user actions. The recommendation task is decomposed into modeling users’ long-term and short-term intentions.

The process of comprehending long-term user interests is accomplished by training an end-to-end embedding model, referred to as PinnerFormer, to learn from a user’s historical actions on the platform. These actions are subsequently transformed into user embeddings, which are designed for optimization based on anticipated long-term future user activities.

This procedure employs an adapted transformer model to operate on users’ sequential actions with the intent to forecast their long-term future activities. Each user’s activity is compiled into a sequence, encompassing their actions over a specific time window, such as one year. The graph neural network–based (GNN-based) PinnerSage embeddings, in conjunction with relevant metadata (for example, the type of action, the timestamp, and so forth), are used to add features to each action in the sequence.

Distinct from traditional sequential modeling tasks and sequential recommendation systems, PinnerFormer is designed to predict extended future user activities rather than the immediately subsequent action. This objective is achieved by training the model to foresee a user’s positive future interactions over a window of 14 days following the embedding’s generation. In comparison, traditional sequential models would anticipate only the subsequent action.

This alternate approach allows for the embedding generation to occur offline in a batch-processing mode, resulting in significant reductions in

infrastructure needs. In contrast to most traditional sequential modeling systems, which operate in real time and incur substantial computational and infrastructure costs, these embeddings can be produced in batches (for instance, on a daily basis) rather than every time a user performs an action.

A dense all-action loss is introduced in this methodology to facilitate batch training of the model. The objective here is not to predict the immediate next action but rather all the actions the user will undertake over the subsequent k days. The aim is to predict all occurrences of a user's positive interactions at intervals such as $T + 3$, $T + 8$, and $T + 12$, thereby compelling the system to learn long-term intentions. While traditionally the last action's embedding is used to make the prediction, the dense all-action loss employs randomly selected positions in the action sequence, and the corresponding embedding is used to predict all actions for each of those positions.

Based on offline and online experimental results, the use of dense all-action loss to train for long-term user actions has significantly bridged the gap between batch generation and real-time generation of user embeddings. Moreover, to accommodate users' short-term interests, the transformer model retrieves the most recent actions for each user in real time, processing them along with the long-term user embeddings.

Summary

Transformers and sequential recommendation systems are really at the cutting edge of modern recommenders. These days, most research in recommendation systems is in the area of sequential datasets, and the hottest recommenders are using longer and longer sequences for prediction. Two important projects are worthy of attention:

Transformers4Rec

This open source project is geared toward scalable transformer models by the NVIDIA Merlin team. For more details, see

[“Transformers4Rec: Bridging the Gap Between NLP and](#)

[Sequential/Session-Based Recommendation](#)” by Gabriel de Souza

Pereira Moreira et al.

Monolith

Also known as the TikTok For You page recommender, this is one of the most popular and exciting recommendation systems at this time. It is a fundamentally sequential recommender, with some elegant hybrid approaches. [“Monolith: Real-Time Recommendation System with Collisionless Embedding Table”](#) by Zhuoran Liu et al. covers the architectural considerations.

Our final step before this book concludes is to consider a few approaches to recommendations. These don’t build exactly on top of what we’ve done but will use some of what we’ve done and introduce a few new ideas.

Let’s sprint to the finish!